

Lab 1 — Instructor Solution and Marking Guide

Cybersecurity Architecture (ACS) · Chapter 1 · Not for student distribution

Lab 1 — Instructor Solution and Marking Guide

1. Pre-lab checklist
2. Timing plan (90 minutes)
3. Reference capture
4. Model answers — in-lab tasks
5. Model answers — challenge questions
6. Marking notes and common failure modes

Lab 1 — Instructor Solution and Marking Guide

Course: Cybersecurity Architecture (ACS) — Chapter 1 **Companion to:** Lab1_Wireshark_Traffic_Analysis.md **Not for distribution to students.**

1. Pre-lab checklist

Run this before the session. It catches the three failures that eat the first twenty minutes of every Wireshark lab.

```
# 1. Wireshark present and non-root capture enabled
wireshark --version | head -1
getcap "$(which dumpcap)"      # expect: cap_net_admin,cap_net_raw=eip

# 2. A plain-HTTP target is reachable from the lab network
curl -sSI http://neverssl.com/ | head -1      # expect: HTTP/1.1 200 OK

# 3. DNS is not being intercepted by a proxy that hides UDP/53
dig +short neverssl.com
```

If the campus network blocks outbound port 80, students will complete Parts 1-3 but fail Part 4. Two fallbacks, in order of preference:

- Stand up a local target: `python3 -m http.server 80` on the instructor machine, and have students `curl http://<instructor-ip>/`. Capturing against a host on the lab segment also makes the ARP exchange visible, which is a bonus.
- Have students use `chapter1_reference.pcap` for Part 4 only.

If `neverssl.com` is unreachable or has changed behaviour, substitute any of: `http://httpforever.com/`, `http://info.cern.ch/`, or a local server as above. Verify on the day — plain-HTTP public endpoints disappear over time.

2. Timing plan (90 minutes)

Minutes	Part	Instructor action
0-10	Part 0	Setup. Circulate — this is where the group permission problem surfaces.
10-25	Part 1	Demo the three panes on the projector. Labour the capture/display filter distinction; it is examinable.
25-40	Part 2	ICMP. Fast part — most students finish early.
40-60	Part 3	DNS. The cache flush is the common blocker.
60-85	Part 4	HTTP. Do Step 4.5 as a live demo first, then let them repeat it.
85-90	Part 5	Export and submission check.

The moment that lands: Step 4.5, the cleartext password. Project it. Then immediately repeat the same request over HTTPS and show `tls` — the contrast, seen back to back, is what students remember from Chapter 1.

3. Reference capture

`chapter1_reference.pcap` — 32 frames, 5.67 s, generated by `make_reference_capture.py`.

It is **synthetic**: assembled frame by frame in Python, not sniffed from a wire. That is deliberate — shipping a real capture in a public course repo means shipping somebody's real traffic. All checksums, sequence numbers, ACK arithmetic and timings are valid, so Wireshark dissects, reassembles and *Follow Streams* it identically to a live capture. Verified with:

```
tshark -r chapter1_reference.pcap \
  -o ip.check_checksum:TRUE -o tcp.check_checksum:TRUE \
  -o udp.check_checksum:TRUE \
  -Y 'ip.checksum.status=="Bad" || tcp.checksum.status=="Bad" || udp.checksum.status=="Bad" ||
      icmp.checksum.status=="Bad"'
# -> no output
```

To regenerate or modify it (e.g. change the credentials, add packet loss):

```
python3 make_reference_capture.py chapter1_reference.pcap
```

Contents

Frames	Content
1-2	ARP who-has 192.168.56.1 / is-at
3-4	DNS A? neverssl.com → 34.223.124.45
5-12	ICMP echo request/reply ×4
13-22	TCP stream 0 — handshake, GET / , 200 OK , teardown
23-32	TCP stream 1 — handshake, POST /login.php , 302 Found , teardown

Values a student using the reference file should report

Field	Value
Client IP / MAC	192.168.56.101 / 08:00:27:aa:bb:cc
Gateway + DNS server	192.168.56.1
Resolved address	34.223.124.45
DNS Transaction ID	0x1a2b (identical in both frames)
DNS client source port	51514
DNS answer TTL	60 s
dns.time	0.0186 s
ICMP Identifier	0x2f01 (12033 decimal)
ICMP sequence numbers	1, 2, 3, 4
Request TTL / reply TTL	64 / 52 → 12 hops
ICMP frame length	98 bytes
HTTP GET response	200 OK, Content-Length: 457
HTTP POST body	username=student&password=Vives2026!&submit=Log+in
POST response	302 Found, Set-Cookie: PHPSESSID=...

Credentials are recoverable three ways — all should be demonstrated:

```
# 1. Packet details pane: expand "HTML Form URL Encoded"
# 2. Tools > Credentials
# 3. Command line:
tshark -r chapter1_reference.pcap -Y 'http.request.method=="POST"' \
      -T fields -e urlencoded-form.key -e urlencoded-form.value
# -> username,password,submit      student,Vives2026!,Log in
```

4. Model answers — in-lab tasks

Task 1.1 — OSI layer mapping

Wireshark tree line	OSI layer
Frame N: ... bytes on wire	— (capture metadata, not a protocol layer)
Ethernet II	Layer 2 — Data Link
Internet Protocol Version 4	Layer 3 — Network
Transmission Control Protocol	Layer 4 — Transport
Hypertext Transfer Protocol	Layer 7 — Application

Mark for: recognising that *Frame* is metadata added by the capture library, not a protocol on the wire. Also accept students who note that OSI layers 5 and 6 (Session, Presentation) have no distinct representation in the TCP/IP stack — that is a correct and perceptive observation, not an error.

Task 1.2 — When a capture filter is the right choice

Full marks for any answer identifying a scenario where the **volume discarded matters**, e.g.:

- Long-running capture on a busy link where storing everything would exhaust disk.
- Capturing on a SPAN/mirror port carrying gigabits — the kernel cannot keep up and will drop packets indiscriminately; a BPF filter drops them *selectively* instead.
- **Legal/privacy scoping:** capturing only traffic to/from an authorised host, so third-party traffic is never written to disk in the first place. This is the strongest answer — flag it for discussion.

A display filter cannot help in any of these because the packets have already been captured, written and stored by the time it applies.

Task 2.1 — ICMP comparison table

Expected shape (values from the reference capture):

Field	Echo request	Echo reply
Source IP	192.168.56.101	34.223.124.45
Destination IP	34.223.124.45	192.168.56.101
ICMP Type	8	0
ICMP Code	0	0
Identifier	0x2f01	0x2f01 (same)
Sequence number	1	1 (same)
TTL	64	52
Frame length	98	98

Mark for: noticing that Identifier and Sequence are **preserved** in the reply — that is the mechanism by which `ping` matches replies to requests, exactly analogous to the DNS Transaction ID in Part 3. Draw that parallel explicitly if nobody does.

Task 2.2 — TTL and hop count

Reply TTL 52, initial value almost certainly 64 → **12 hops**.

Reasoning: the responder set TTL to its own OS default (64 for Linux, 128 for Windows, 255 for many network devices), and each router decremented it by one. $64 - 52 = 12$.

Accept disagreement with `traceroute` as correct, provided the student gives a valid reason. Good reasons:

- The return path differs from the forward path (routing is not symmetric); `traceroute` measures the *forward* path, TTL measures the *return* path.
- The initial TTL was assumed, not observed — if the responder starts at 255, the hop count is 203, not 12.
- Load balancers / ECMP send successive packets via different paths.
- Some middleboxes rewrite TTL.

This is the question that separates students who reasoned from students who subtracted.

Task 2.3 — Why echo the payload back

The payload is echoed so the sender can verify the data returned **intact** — it detects corruption, and lets `ping` compute round-trip time by embedding a timestamp in the payload it will get back.

Security consequence: ICMP echo carries **arbitrary attacker-controlled data in both directions** and is very often permitted through firewalls that block everything else. That makes it a covert channel — **ICMP tunnelling** (tools: `ptunnel`, `icmpsh`, `iodine` for the DNS equivalent). Data exfiltration or C2 traffic is encoded in the payload field.

Full marks require both halves: *why the protocol does it* and *how it is abused*.

Task 3.1 — DNS field table

See the reference values table in §3 above. For live captures, the DNS server will be the site resolver and the resolved address for `neverssl.com` varies — do not mark a specific IP as wrong.

Mark for: the Transaction ID being identical in query and response, and the client source port being a high random port (ephemeral range, ~32768-60999 on Linux). If a student reports a *fixed* or low source port, their resolver is unusual — worth a look, it may indicate a proxy.

Task 3.2 — DNS security

1. **Confidentiality.** The observer learns every domain the client resolves — a near-complete browsing history, even when the sites themselves use HTTPS. Accept students who also argue **integrity**, since nothing prevents modification in flight; give credit if they justify it.
2. **DNS cache poisoning / DNS spoofing** (Kaminsky attack is the canonical named form). The attacker forges a response with the matching Transaction ID and the correct source/destination ports, and wins the race against the legitimate server. The victim's resolver caches the forged answer for the TTL.
3. Two protocols, and this is the part students get wrong — the expected answer distinguishes **channel encryption** from **data authentication**:

Protocol	What it actually protects
DoH (DNS over HTTPS, RFC 8484) or DoT (DNS over TLS, RFC 7858)	Confidentiality + integrity of the channel between client and resolver. Stops on-path eavesdropping and tampering. Does not authenticate the data itself — a malicious or compromised resolver can still lie.
DNSSEC (RFC 4033-4035)	Authenticity + integrity of the data , via a chain of cryptographic signatures from the root. Proves the answer came from the zone owner. Does not encrypt — anyone on the path still sees the query.

Full marks require the student to state that these solve different problems and are complementary. A student who lists both but treats them as interchangeable gets partial credit only. This is the single most valuable discussion point in Part 3.

Task 4.1 — Three-way handshake

Expected: SYN (seq=X), SYN-ACK (seq=Y, ack=X+1), ACK (seq=X+1, ack=Y+1).

In the reference capture, absolute ISNs are 0x5C2A1000 (client) and 0x91B33000 (server) for stream 0. Live captures will differ — ISNs are randomised, and that randomisation is itself a security control (it is what makes blind TCP sequence prediction hard).

Mark for: the student actually turning off relative sequence numbers and reporting real values. If they report Seq=0, they did not do the step.

Task 4.2 — Follow HTTP Stream / reassembly

Wireshark performed **TCP stream reassembly**: it identified all segments belonging to the same connection (same 4-tuple of source IP, source port, destination IP, destination port), ordered them by sequence number, removed retransmissions and overlaps, stripped the TCP headers and concatenated the payloads into the two byte-streams that make up the conversation.

Credit any answer conveying *ordering by sequence number + concatenating payloads across multiple segments*. Bonus credit for mentioning that Wireshark also un-chunks Transfer-Encoding: chunked and decompresses Content-Encoding: gzip at the HTTP layer, which is a separate step above reassembly.

Task 4.3 — Cleartext credentials

1. **Confidentiality**, broken at **Layer 7 (Application)** — HTTP transmits the form body as plain ASCII. Accept "exposed at every layer below 7 as well", since the bytes are readable at layers 2, 3 and 4 too; the *cause* is the Layer 7 protocol's lack of encryption.
2. Over HTTPS the credentials are **not** readable — after the TLS handshake the filter `tls` shows only `Application Data` records of opaque ciphertext.

Still visible: the **SNI** in the `Client Hello`. SNI is sent *before* the encrypted channel exists, because the server needs to know which certificate to present — a chicken-and-egg problem. So the hostname leaks even though

everything after it does not. (Follow-up if the group is strong: ECH — Encrypted Client Hello — is the fix, and it depends on DoH to fetch the public key, which ties back to Part 3.)

3. Things HTTPS does **not** hide, any of:

- Destination IP address and port
- The SNI hostname
- Traffic volume, packet sizes and timing (enough for traffic-analysis attacks that fingerprint which page was visited)
- The fact that a connection happened at all, and its duration
- The server's certificate, in TLS 1.2 and earlier

5. Model answers — challenge questions

1. Capture filter applies in the kernel before packets are stored, uses BPF syntax, and discards non-matching traffic permanently. Display filter applies to an already-captured file, uses Wireshark syntax, and only hides packets.

Capture: `icmp and host 8.8.8.8` — Display: `icmp && ip.addr == 8.8.8.8`

2. ICMP is a Layer 3 (Network) protocol carried directly in IP with protocol number 1. Ports are a Layer 4 (Transport) construct used to multiplex between processes; ICMP is not a transport for application data, so it has no need for them — it is demultiplexed by Type/Code instead.

3. TTL is decremented by one at each router hop. The request's TTL is the sender's OS default as it leaves; the reply's TTL is the *responder's* default minus the hops on the return path.

4. The Transaction ID is a 16-bit value the client generates and the server copies, binding a response to its query. Abuse: an off-path attacker who guesses it (only 65,536 values; fewer in practice with a weak PRNG) and wins the race against the real server gets their forged answer cached — DNS cache poisoning. Source-port randomisation was added to widen the entropy, and DNSSEC to fix it properly.

5. DNS switches to TCP when the response exceeds the UDP payload limit (512 bytes classically, or the EDNS0-advertised size, commonly 1232 or 4096) and the truncation `TC` bit is set — the client retries over TCP. Also always for zone transfers (AXFR/IXFR), and for DoT/DoH which are TCP-based by construction.

6. SYN → SYN-ACK → ACK. Two-way is insufficient because each side must both *send* its own initial sequence number and *receive confirmation* that the other side got it. After two messages the client knows the server received its ISN, but the server has no confirmation that the client received *the server's* ISN. The third packet supplies that. It also establishes that both directions of the path work, and that the client can receive at its claimed address — which is what makes blind spoofing hard.

7. `tcp.stream eq 0` selects every packet belonging to the first TCP connection Wireshark saw in the file. Wireshark assigns a stream index to each unique 4-tuple (source IP, source port, destination IP, destination port) and tracks the connection's lifetime from SYN to FIN/RST.

8. Exposed at Layer 7 (Application) — HTTP has no encryption. HTTPS fixes it by inserting TLS between Layer 4 and Layer 7, usually described as Layer 5/6 (Session/Presentation) in OSI terms. Accept "between transport and application" as the substance of the answer; do not penalise students for the well-known ambiguity of where TLS sits in the OSI model — but do expect them to notice it is *not* at Layer 7 where the problem was.

9. Destination IP and port; SNI hostname in the Client Hello; packet sizes, timing and volume; connection duration; the certificate (pre-TLS 1.3). Any three.

10. `tshark` — it is scriptable, runs headless, and handles large files and continuous capture with ring buffers, which the GUI does not.

The legal constraint is the decisive half of this answer: doing this across a campus network means **intercepting third parties' communications**. Under Belgian law (Art. 314bis Sw.) and the EU ePrivacy Directive this is unlawful without a specific legal basis; GDPR additionally applies because packet contents and IP addresses are personal data. In practice it requires documented authorisation, a defined and minimised scope, an acceptable-use policy that users have been informed of, data-retention limits, and normally sign-off from the DPO. **Technical capability is not authorisation** — that is the sentence to look for.

Award full marks only if the student names both the tool *and* a substantive legal constraint. Naming `tshark` alone is half the answer.

6. Marking notes and common failure modes

Observed	Interpretation	Action
Screenshots show Seq=0 , Ack=1	Relative sequence numbers still on; Task 4.1 not done	Deduct within <i>HTTP analysis</i>
Report has no display filter visible in any screenshot	Student browsed manually instead of filtering	Deduct <i>Filter usage</i> in full
Capture contains Slack/ Teams/mail traffic	Did not filter before exporting	Deduct <i>Report quality</i> ; raise the privacy point with them directly
Capture contains a real personal credential	Ignored the warning in Step 4.5	Do not mark. Delete the file, tell the student, ask for a resubmission.
TTL answer is exactly 64 - reply_ttl with no discussion	Subtracted without reasoning	Partial credit on Task 2.2
DoH and DNSSEC listed as alternatives to each other	The standard misconception	Partial credit on Task 3.2; correct it in the plenary
Identical screenshots across several students	Shared capture file	Check capinfos SHA256 — the reference file's is 9cd3df2b2eb64197bdee9c8ac6e46066ff1dc0a4e7d56ecccc12694001f449d7 . A student's live capture must not match it.

Quick submission check

```
#!/bin/bash
# Verify a submitted capture contains all three required protocols.
f="$1"
for proto in icmp dns http; do
  n=$(tshark -r "$f" -Y "$proto" 2>/dev/null | wc -l)
  printf "%-6s %s packets %s\n" "$proto" "$n" "${[ "$n" -gt 0 ] && echo OK || echo MISSING}"
done
echo -n "SHA256: "; shasum -a 256 "$f" | cut -d' ' -f1
```

Author: Milan Dima — milan.dima@vives.be **Last updated:** 30 September 2026